

Métodos de Machine Learning para Clasificación

Edison Achalma

Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Palabras Claves: keyword1, keyword2

Tabla de contenidos

Introduction	3
1 Introducción al aprendizaje supervisado	3
1.1 Concepto fundamental	3
1.2 Tipos de problemas de clasificación	3
1.3 Dataset de ejemplo: Iris	4
2 Regresión logística	5
2.1 Fundamento matemático	5
2.2 Implementación	6
2.3 Predicción e interpretación	6
2.4 Visualización del modelo	7
3 K-Nearest Neighbors (KNN)	8
3.1 Fundamento del algoritmo	8
3.2 Implementación	9
3.3 Visualización del efecto de k	10
3.4 Visualización de fronteras de decisión	11
4 Árboles de decisión	11
4.1 Fundamento del algoritmo	11
4.2 Implementación básica	12

Edison Achalma  <https://orcid.org/0000-0001-6996-3364>

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; <https://credit.niso.org/>) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

4.3	Visualización del árbol	13
4.4	Efecto de la profundidad	13
5	Random Forests	15
5.1	Fundamento del algoritmo	15
5.2	Implementación	16
5.3	Efecto del número de árboles	16
6	Clasificación multi-clase	17
6.1	Dataset con 3 clases	17
6.2	Entrenar modelos multi-clase	18
6.3	Visualización multi-clase	19
7	Comparación de modelos	20
7.1	Curvas ROC para cada modelo	20
7.2	Tabla comparativa de métricas	21
8	Aplicaciones económicas	22
8.1	Aplicación 1: Clasificación de riesgo crediticio	22
8.2	Aplicación 2: Segmentación de clientes	24
9	Ejercicios prácticos	26
9.1	Ejercicio: Optimización de hiperparámetros	26
10	Conclusión	28
11	Próximos pasos	28
12	Recursos adicionales	28
13	Publicaciones Similares	29

Métodos de Machine Learning para Clasificación

En esta novena guía exploraremos los principales algoritmos de machine learning para problemas de clasificación. Estos métodos son fundamentales para resolver problemas económicos como clasificación de riesgo crediticio, predicción de comportamiento de consumidores, segmentación de mercados, entre otros.

1 Introducción al aprendizaje supervisado

1.1 Concepto fundamental

El aprendizaje supervisado es un tipo de machine learning donde el modelo aprende de datos etiquetados (con respuestas conocidas) para predecir etiquetas de nuevos datos.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.colors import ListedColormap

# Scikit-learn
from sklearn.datasets import load_iris, make_classification
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_curve, auc, accuracy_score)

# Modelos
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# Visualización de árboles
from sklearn import tree

# Configuración
plt.style.use('seaborn-v0_8-darkgrid')
plt.rcParams['figure.figsize'] = (12, 6)
np.random.seed(42)

print("Librerías importadas exitosamente")
```

1.2 Tipos de problemas de clasificación

```
print("""
TIPOS DE PROBLEMAS DE CLASIFICACIÓN

1. CLASIFICACIÓN BINARIA
```

```

- Dos clases posibles
- Ejemplos económicos:
  * Aprobado/Rechazado (crédito)
  * Paga/No paga (incumplimiento)
  * Éxito/Fracaso (empresa)
  * Compra/No compra (marketing)

2. CLASIFICACIÓN MULTI-CLASE
  - Más de dos clases
  - Ejemplos económicos:
    * Clasificación de riesgo (AAA, AA, A, BBB, etc.)
    * Segmentación de clientes (bajo, medio, alto valor)
    * Categorías de productos
    * Niveles socioeconómicos (A, B, C, D, E)

3. CLASIFICACIÓN MULTI-ETIQUETA
  - Múltiples etiquetas simultáneas
  - Ejemplos económicos:
    * Características de consumidor (múltiples intereses)
    * Atributos de producto (múltiples categorías)
"""

```

1.3 Dataset de ejemplo: Iris

```

# Cargar dataset clásico
iris = load_iris()

# Crear DataFrame
df_iris = pd.DataFrame(
    data=iris.data,
    columns=iris.feature_names
)
df_iris['especie'] = iris.target
df_iris['especie_nombre'] = df_iris['especie'].map({
    0: 'Setosa',
    1: 'Versicolor',
    2: 'Virginica'
})

print("Dataset Iris:")
print(df_iris.head())
print(f"\nForma: {df_iris.shape}")
print(f"\nDistribución de especies:")
print(df_iris['especie_nombre'].value_counts())

# Visualizar (usar solo 2 características)
plt.figure(figsize=(10, 6))

```

```

for especie in df_iris['especie'].unique():
    mask = df_iris['especie'] == especie
    plt.scatter(
        df_iris.loc[mask, 'sepal length (cm)'],
        df_iris.loc[mask, 'sepal width (cm)'],
        label=df_iris.loc[mask, 'especie_nombre'].iloc[0],
        alpha=0.7,
        s=100
    )

plt.xlabel('Largo del sépallo (cm)')
plt.ylabel('Ancho del sépallo (cm)')
plt.title('Dataset Iris - Clasificación de especies')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('iris_scatter.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'iris_scatter.png'")

```

2 Regresión logística

2.1 Fundamento matemático

La regresión logística es un modelo lineal para clasificación que estima probabilidades usando la función logística (sigmoide).

```

print("""
REGRESIÓN LOGÍSTICA

Modelo matemático:
    y =  + x + x + ... + x +

    p = 1 / (1 + e^(-y))

Donde:
- y: Combinación lineal de características
- p: Probabilidad de pertenecer a la clase positiva
- : Coeficientes (pesos) del modelo
- e: Base del logaritmo natural (2.718)

Características:
Simple e interpretable
Rápido de entrenar
Funciona bien con características linealmente separables
Proporciona probabilidades calibradas
Asume relación lineal
Sensible a outliers

Aplicaciones económicas:

```

```
- Predicción de incumplimiento crediticio
- Clasificación de riesgo empresarial
- Predicción de compra de producto
- Churn (abandono de clientes)
""")
```

2.2 Implementación

```
# Preparar datos (clasificación binaria: Virginica vs resto)
X = iris.data[:, :2] # Solo usar 2 características para visualización
y = (iris.target == 2).astype(int) # 1 si es Virginica, 0 si no

# Dividir datos
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

print(f"Conjunto de entrenamiento: {X_train.shape[0]} muestras")
print(f"Conjunto de prueba: {X_test.shape[0]} muestras")
print(f"\nDistribución en entrenamiento:")
print(f"  Clase 0 (No Virginica): {(y_train == 0).sum()}")
print(f"  Clase 1 (Virginica): {(y_train == 1).sum()}")

# Entrenar modelo
modelo_logit = LogisticRegression(random_state=42)
modelo_logit.fit(X_train, y_train)

print(f"\n\nMODELO ENTRENADO")
print("=" * 70)
print(f"Intercepto ( ): {modelo_logit.intercept_[0]:.4f}")
print(f"Coeficientes:")
print(f"    (largo sépalo): {modelo_logit.coef_[0, 0]:.4f}")
print(f"    (ancho sépalo): {modelo_logit.coef_[0, 1]:.4f}")
```

2.3 Predicción e interpretación

```
# Predecir probabilidades
probs_train = modelo_logit.predict_proba(X_train)[:, 1]
probs_test = modelo_logit.predict_proba(X_test)[:, 1]

# Predecir clases
y_pred = modelo_logit.predict(X_test)

# Evaluar
accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy en test: {accuracy:.4f} ({accuracy*100:.1f}%)")
```

```

# Matriz de confusión
cm = confusion_matrix(y_test, y_pred)
print(f"\nMatriz de confusión:")
print(cm)

# Reporte de clasificación
print(f"\nReporte de clasificación:")
print(classification_report(y_test, y_pred,
                           target_names=['No Virginica', 'Virginica']))

# Ejemplo de predicción
ejemplo = np.array([[6.5, 3.0]]) # Nuevo ejemplo
prob = modelo_logit.predict_proba(ejemplo)[0, 1]
pred = modelo_logit.predict(ejemplo)[0]

print(f"\n\nEJEMPLO DE PREDICCIÓN")
print(f"Características: Largo={ejemplo[0, 0]} cm, Ancho={ejemplo[0, 1]} cm")
print(f"Probabilidad de ser Virginica: {prob:.4f} ({prob*100:.1f}%)")
print(f"Predicción: {'Virginica' if pred == 1 else 'No Virginica'}")

```

2.4 Visualización del modelo

```

def plot_decision_boundary(modelo, X, y, title, filename):
    """Visualiza la frontera de decisión del modelo"""

    h = 0.02 # Tamaño del paso en la malla

    # Crear malla
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                         np.arange(y_min, y_max, h))

    # Predecir para cada punto en la malla
    Z = modelo.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:, 1]
    Z = Z.reshape(xx.shape)

    # Graficar
    plt.figure(figsize=(12, 8))

    # Contorno de probabilidades
    contour = plt.contourf(xx, yy, Z, levels=20, cmap='RdYlBu_r', alpha=0.6)
    plt.colorbar(contour, label='Probabilidad clase positiva')

    # Frontera de decisión (probabilidad = 0.5)
    plt.contour(xx, yy, Z, levels=[0.5], colors='black', linewidths=2)

```

```

# Puntos de datos
scatter = plt.scatter(X[:, 0], X[:, 1], c=y,
                      cmap='RdYlBu_r', edgecolors='black',
                      s=100, alpha=0.9)

plt.xlabel('Largo del sépallo (cm)', fontsize=12)
plt.ylabel('Ancho del sépallo (cm)', fontsize=12)
plt.title(title, fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(filename, dpi=300, bbox_inches='tight')
print(f"Gráfico guardado como '{filename}'")

# Visualizar
plot_decision_boundary(
    modelo_logit, X_train, y_train,
    'Regresión Logística - Frontera de Decisión',
    'logistic_regression_boundary.png'
)

```

3 K-Nearest Neighbors (KNN)

3.1 Fundamento del algoritmo

```

print("""
K-NEAREST NEIGHBORS (KNN)

Principio:
"Dime quiénes son tus vecinos y te diré quién eres"

Funcionamiento:
1. Para clasificar un nuevo punto:
    a. Calcula distancia a todos los puntos de entrenamiento
    b. Selecciona los K vecinos más cercanos
    c. Vota por mayoría: clase más común entre vecinos

Parámetros clave:
- k: Número de vecinos a considerar
  * k pequeño: Más flexible, sensible a ruido
  * k grande: Más suave, puede perder detalles

Métricas de distancia:
- Euclidiana:  $\sqrt{\sum (x_i - y_i)^2}$ 
- Manhattan:  $\sum |x_i - y_i|$ 
- Minkowski:  $(\sum |x_i - y_i|^p)^{1/p}$ 

```


Características:

- No paramétrico (no asume distribución)
- Simple de entender
- Funciona bien con fronteras complejas
- Lento en predicción con datasets grandes
- Sensible a la escala de características
- Sufre con alta dimensionalidad

Aplicaciones económicas:

- Sistema de recomendación de productos
- Clasificación de clientes similares
- Detección de anomalías en transacciones
- Valuación de propiedades (casas similares)

```
""")
```

3.2 Implementación

```
# Estandarizar características (importante para KNN)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Entrenar modelos con diferentes valores de k
k_values = [1, 3, 5, 10, 15, 20]
resultados_knn = []

print("EVALUACIÓN DE KNN CON DIFERENTES VALORES DE K")
print("=" * 70)
print(f"{'K':<5} {'Accuracy Train':<15} {'Accuracy Test':<15} {'Diferencia':<12}")
print("-" * 70)

for k in k_values:
    # Entrenar modelo
    modelo_knn = KNeighborsClassifier(n_neighbors=k)
    modelo_knn.fit(X_train_scaled, y_train)

    # Evaluar
    acc_train = modelo_knn.score(X_train_scaled, y_train)
    acc_test = modelo_knn.score(X_test_scaled, y_test)
    diff = acc_train - acc_test

    resultados_knn.append({
        'k': k,
        'acc_train': acc_train,
        'acc_test': acc_test,
        'diff': diff
    })
```

```

    print(f"{k:<5} {acc_train:<15.4f} {acc_test:<15.4f} {diff:<12.4f}")

# Encontrar mejor k
df_knn = pd.DataFrame(resultados_knn)
mejor_k = df_knn.loc[df_knn['acc_test'].idxmax(), 'k']
print(f"\n\nMejor k: {int(mejor_k)} (mayor accuracy en test)")

```

3.3 Visualización del efecto de k

```

# Entrenar modelo con mejor k
modelo_knn_final = KNeighborsClassifier(n_neighbors=int(mejor_k))
modelo_knn_final.fit(X_train_scaled, y_train)

# Graficar accuracy vs k
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Accuracy
axes[0].plot(df_knn['k'], df_knn['acc_train'], 'o-',
             label='Training', linewidth=2, markersize=8)
axes[0].plot(df_knn['k'], df_knn['acc_test'], 's-',
             label='Test', linewidth=2, markersize=8)
axes[0].axvline(x=mejor_k, color='red', linestyle='--',
               label=f'Mejor k={int(mejor_k)}')
axes[0].set_xlabel('Número de vecinos (k)')
axes[0].set_ylabel('Accuracy')
axes[0].set_title('Accuracy vs K')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Overfitting
axes[1].plot(df_knn['k'], df_knn['diff'], 'o-',
             linewidth=2, markersize=8, color='purple')
axes[1].axhline(y=0, color='black', linestyle='--', alpha=0.3)
axes[1].axvline(x=mejor_k, color='red', linestyle='--',
               label=f'Mejor k={int(mejor_k)}')
axes[1].set_xlabel('Número de vecinos (k)')
axes[1].set_ylabel('Train Accuracy - Test Accuracy')
axes[1].set_title('Diferencia Train-Test (Indicador de Overfitting)')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('knn_k_selection.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'knn_k_selection.png'")

```

3.4 Visualización de fronteras de decisión

```
# Comparar diferentes valores de k
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes = axes.ravel()

h = 0.02
x_min, x_max = X_train_scaled[:, 0].min() - 1, X_train_scaled[:, 0].max() + 1
y_min, y_max = X_train_scaled[:, 1].min() - 1, X_train_scaled[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))

for idx, k in enumerate([1, 3, 5, 10, 15, 20]):
    # Entrenar modelo
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)

    # Predecir
    Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Graficar
    axes[idx].contourf(xx, yy, Z, alpha=0.4, cmap='RdYlBu_r')
    axes[idx].scatter(X_train_scaled[:, 0], X_train_scaled[:, 1],
                     c=y_train, cmap='RdYlBu_r',
                     edgecolors='black', s=50)

    acc = knn.score(X_test_scaled, y_test)
    axes[idx].set_title(f'k={k}, Accuracy={acc:.3f}')
    axes[idx].set_xlabel('Feature 1 (escalado)')
    axes[idx].set_ylabel('Feature 2 (escalado)')
    axes[idx].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('knn_comparison.png', dpi=300, bbox_inches='tight')
print("Comparación guardada como 'knn_comparison.png'")
```

4 Árboles de decisión

4.1 Fundamento del algoritmo

```
print("""
ÁRBOLES DE DECISIÓN (DECISION TREES)

Funcionamiento:
El algoritmo construye un árbol de decisiones mediante:
1. Selección de la mejor característica para dividir datos
```

2. División de datos en subgrupos
3. Repetición recursiva hasta cumplir criterio de parada

Criterios de división:

- Gini Impurity: Mide impureza de un nodo

$$\text{Gini} = 1 - \sum (\pi_i^2)$$
, donde π_i = proporción de clase i
- Entropy (Information Gain): Mide desorden

$$\text{Entropy} = -\sum (\pi_i * \log_2(\pi_i))$$

Parámetros importantes:

- max_depth: Profundidad máxima del árbol
- min_samples_split: Mínimo de muestras para dividir nodo
- min_samples_leaf: Mínimo de muestras en hoja
- max_features: Número de características a considerar

Ventajas:

Fácil de interpretar y visualizar
 No requiere normalización de datos
 Maneja características numéricas y categóricas
 Captura relaciones no lineales
 Puede manejar missing values

Desventajas:

Propenso a overfitting
 Inestable (pequeños cambios → árbol diferente)
 Puede crear árboles sesgados con clases desbalanceadas

Aplicaciones económicas:

- Aprobación de créditos (reglas claras)
 - Segmentación de clientes
 - Detección de fraude
 - Valuación de riesgo
 - Decisiones de inversión
- """)

4.2 Implementación básica

```
# Entrenar árbol de decisión
modelo_tree = DecisionTreeClassifier(
    max_depth=3,
    min_samples_split=10,
    min_samples_leaf=5,
    random_state=42
)

modelo_tree.fit(X_train, y_train)
```

```
# Evaluar
y_pred_tree = modelo_tree.predict(X_test)
acc_tree = accuracy_score(y_test, y_pred_tree)

print("ÁRBOL DE DECISIÓN")
print("=" * 70)
print(f"Profundidad del árbol: {modelo_tree.get_depth()}")
print(f"Número de hojas: {modelo_tree.get_n_leaves()}")
print(f"Accuracy en test: {acc_tree:.4f} ({acc_tree*100:.1f}%)")

# Importancia de características
importancias = pd.DataFrame({
    'caracteristica': ['Largo sépalo', 'Ancho sépalo'],
    'importancia': modelo_tree.feature_importances_
}).sort_values('importancia', ascending=False)

print(f"\nImportancia de características:")
for _, row in importancias.iterrows():
    print(f" {row['caracteristica']}: {row['importancia']:.4f}")
```

4.3 Visualización del árbol

```
# Visualizar árbol
plt.figure(figsize=(20, 10))
tree.plot_tree(
    modelo_tree,
    feature_names=['Largo sépalo', 'Ancho sépalo'],
    class_names=['No Virginica', 'Virginica'],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.title('Árbol de Decisión - Clasificación de Iris',
          fontsize=16, fontweight='bold', pad=20)
plt.tight_layout()
plt.savefig('decision_tree.png', dpi=300, bbox_inches='tight')
print("\nÁrbol visualizado y guardado como 'decision_tree.png'")
```

4.4 Efecto de la profundidad

```
# Evaluar diferentes profundidades
profundidades = range(1, 11)
resultados_depth = []

print("\n\nEFECTO DE LA PROFUNDIDAD DEL ÁRBOL")
print("=" * 70)
```

```
print(f"{'Profundidad':<12} {'Accuracy Train':<15} {'Accuracy Test':<15} {'Hojas':<10}")
print("-" * 70)

for depth in profundidades:
    tree_model = DecisionTreeClassifier(max_depth=depth, random_state=42)
    tree_model.fit(X_train, y_train)

    acc_train = tree_model.score(X_train, y_train)
    acc_test = tree_model.score(X_test, y_test)
    n_hojas = tree_model.get_n_leaves()

    resultados_depth.append({
        'depth': depth,
        'acc_train': acc_train,
        'acc_test': acc_test,
        'n_leaves': n_hojas
    })

    print(f"{depth:<12} {acc_train:<15.4f} {acc_test:<15.4f} {n_hojas:<10}")

# Graficar
df_depth = pd.DataFrame(resultados_depth)

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Accuracy
axes[0].plot(df_depth['depth'], df_depth['acc_train'],
             'o-', label='Training', linewidth=2)
axes[0].plot(df_depth['depth'], df_depth['acc_test'],
             's-', label='Test', linewidth=2)
axes[0].set_xlabel('Profundidad máxima')
axes[0].set_ylabel('Accuracy')
axes[0].set_title('Accuracy vs Profundidad del Árbol')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Complejidad
axes[1].plot(df_depth['depth'], df_depth['n_leaves'],
             'o-', color='green', linewidth=2)
axes[1].set_xlabel('Profundidad máxima')
axes[1].set_ylabel('Número de hojas')
axes[1].set_title('Complejidad del Árbol')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('tree_depth_analysis.png', dpi=300, bbox_inches='tight')
print("\nAnálisis guardado como 'tree_depth_analysis.png'")
```

5 Random Forests

5.1 Fundamento del algoritmo

```
print("""
RANDOM FORESTS (BOSQUES ALEATORIOS)

Concepto:
"La sabiduría de las multitudes"
Ensemble de múltiples árboles de decisión

Funcionamiento:
1. Bootstrap: Crear múltiples muestras aleatorias con reemplazo
2. Para cada muestra:
    a. Entrenar árbol de decisión
    b. En cada división, considerar solo subset aleatorio de características
3. Predicción final: Votación mayoritaria de todos los árboles

Parámetros clave:
- n_estimators: Número de árboles (más = mejor, pero más lento)
- max_depth: Profundidad de cada árbol
- max_features: Características a considerar en cada división
- min_samples_split: Mínimo para dividir nodo
- bootstrap: Usar bootstrap o no

Ventajas:
    Reduce overfitting vs árbol único
    Más robusto y preciso
    Proporciona importancia de características
    Funciona bien out-of-the-box
    Maneja grandes dimensiones

Desventajas:
    Menos interpretable que árbol único
    Más lento para entrenar y predecir
    Requiere más memoria

Aplicaciones económicas:
- Credit scoring (alta precisión)
- Predicción de precios (inmuebles, acciones)
- Detección de fraude
- Churn prediction
- Segmentación avanzada de clientes
""")
```

5.2 Implementación

```
# Entrenar Random Forest
modelo_rf = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    min_samples_split=10,
    min_samples_leaf=5,
    random_state=42,
    n_jobs=-1 # Usar todos los procesadores
)

modelo_rf.fit(X_train, y_train)

# Evaluar
y_pred_rf = modelo_rf.predict(X_test)
acc_rf = accuracy_score(y_test, y_pred_rf)

print("RANDOM FOREST")
print("=" * 70)
print(f"Número de árboles: {modelo_rf.n_estimators}")
print(f"Accuracy en test: {acc_rf:.4f} ({acc_rf*100:.1f}%)")

# Importancia de características
importancias_rf = pd.DataFrame({
    'caracteristica': ['Largo sépalo', 'Ancho sépalo'],
    'importancia': modelo_rf.feature_importances_,
    'std': np.std([tree.feature_importances_
                    for tree in modelo_rf.estimators_], axis=0)
}).sort_values('importancia', ascending=False)

print(f"\nImportancia de características:")
for _, row in importancias_rf.iterrows():
    print(f"  {row['caracteristica']}: {row['importancia']:.4f} (±{row['std']:.4f})")
```

5.3 Efecto del número de árboles

```
# Evaluar diferentes números de árboles
n_trees_list = [1, 5, 10, 20, 50, 100, 200]
resultados_trees = []

print("\n\nEFECTO DEL NÚMERO DE ÁRBOLES")
print("=" * 70)
print(f"{'N Árboles':<12} {'Accuracy Train':<15} {'Accuracy Test':<15}")
print("-" * 70)

for n_trees in n_trees_list:
```



```

rf = RandomForestClassifier(
    n_estimators=n_trees,
    max_depth=5,
    random_state=42
)
rf.fit(X_train, y_train)

acc_train = rf.score(X_train, y_train)
acc_test = rf.score(X_test, y_test)

resultados_trees.append({
    'n_trees': n_trees,
    'acc_train': acc_train,
    'acc_test': acc_test
})

print(f"{n_trees:<12} {acc_train:<15.4f} {acc_test:<15.4f}")

# Graficar
df_trees = pd.DataFrame(resultados_trees)

plt.figure(figsize=(10, 6))
plt.plot(df_trees['n_trees'], df_trees['acc_train'],
         'o-', label='Training', linewidth=2, markersize=8)
plt.plot(df_trees['n_trees'], df_trees['acc_test'],
         's-', label='Test', linewidth=2, markersize=8)
plt.xlabel('Número de árboles')
plt.ylabel('Accuracy')
plt.title('Accuracy vs Número de Árboles en Random Forest')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xscale('log')
plt.tight_layout()
plt.savefig('random_forest_ntrees.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'random_forest_ntrees.png'")

```

6 Clasificación multi-clase

6.1 Dataset con 3 clases

```

# Usar todas las especies de Iris (3 clases)
X_multiclass = iris.data[:, :2]
y_multiclass = iris.target

print("CLASIFICACIÓN MULTI-CLASE")
print("=" * 70)
print(f"Número de clases: {len(np.unique(y_multiclass))}")

```

```
print(f"Clases: {iris.target_names}")
print(f"\nDistribución:")
for i, nombre in enumerate(iris.target_names):
    count = (y_multiclass == i).sum()
    print(f"    {nombre}: {count} muestras")

# Dividir datos
X_train_mc, X_test_mc, y_train_mc, y_test_mc = train_test_split(
    X_multiclass, y_multiclass, test_size=0.3, random_state=42
)
```

6.2 Entrenar modelos multi-clase

```
# Entrenar cada modelo
modelos_mc = {
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=200),
    'KNN': KNeighborsClassifier(n_neighbors=5),
    'Decision Tree': DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
}

resultados_mc = {}

print("\n\nRESULTADOS CLASIFICACIÓN MULTI-CLASE")
print("=" * 70)

for nombre, modelo in modelos_mc.items():
    # Entrenar
    modelo.fit(X_train_mc, y_train_mc)

    # Predecir
    y_pred = modelo.predict(X_test_mc)

    # Evaluar
    acc = accuracy_score(y_test_mc, y_pred)

    resultados_mc[nombre] = {
        'modelo': modelo,
        'accuracy': acc
    }

    print(f"\n{n{nombre}:}")
    print(f"    Accuracy: {acc:.4f} ({acc*100:.1f}%)")

# Matriz de confusión
cm = confusion_matrix(y_test_mc, y_pred)
print(f"    Matriz de confusión:")
```

```

    print(f"    {cm}")

# Encontrar mejor modelo
mejor_modelo_mc = max(resultados_mc, key=lambda x: resultados_mc[x]['accuracy'])
print(f"\n\nMejor modelo: {mejor_modelo_mc}")
print(f"Accuracy: {resultados_mc[mejor_modelo_mc]['accuracy']:.4f}")

```

6.3 Visualización multi-clase

```

# Visualizar fronteras de decisión para cada modelo
fig, axes = plt.subplots(2, 2, figsize=(16, 14))
axes = axes.ravel()

h = 0.02
x_min, x_max = X_multiclass[:, 0].min() - 0.5, X_multiclass[:, 0].max() + 0.5
y_min, y_max = X_multiclass[:, 1].min() - 0.5, X_multiclass[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))

for idx, (nombre, resultado) in enumerate(resultados_mc.items()):
    modelo = resultado['modelo']
    acc = resultado['accuracy']

    # Predecir para malla
    Z = modelo.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Graficar
    axes[idx].contourf(xx, yy, Z, alpha=0.4, cmap='viridis')
    scatter = axes[idx].scatter(X_multiclass[:, 0], X_multiclass[:, 1],
                               c=y_multiclass, cmap='viridis',
                               edgecolors='black', s=50)

    axes[idx].set_xlabel('Largo del sépal (cm)')
    axes[idx].set_ylabel('Ancho del sépal (cm)')
    axes[idx].set_title(f'{nombre}\nAccuracy={acc:.3f}')
    axes[idx].grid(True, alpha=0.3)

# Leyenda común
plt.colorbar(scatter, ax=axes, label='Especie', ticks=[0, 1, 2])

plt.tight_layout()
plt.savefig('multiclass_comparison.png', dpi=300, bbox_inches='tight')
print("\nComparación guardada como 'multiclass_comparison.png'")

```

7 Comparación de modelos

7.1 Curvas ROC para cada modelo

```
# Calcular curvas ROC para clasificación binaria
def calcular_roc_multimodelo(modelos_dict, X_test, y_test):
    """Calcula curvas ROC para múltiples modelos"""

    plt.figure(figsize=(10, 8))

    for nombre, modelo in modelos_dict.items():
        # Obtener probabilidades
        if hasattr(modelo, 'predict_proba'):
            probs = modelo.predict_proba(X_test)[: , 1]
        else:
            probs = modelo.decision_function(X_test)

        # Calcular curva ROC
        fpr, tpr, _ = roc_curve(y_test, probs)
        roc_auc = auc(fpr, tpr)

        # Graficar
        plt.plot(fpr, tpr, linewidth=2,
                 label=f'{nombre} (AUC={roc_auc:.3f})')

    # Línea diagonal
    plt.plot([0, 1], [0, 1], 'k--', linewidth=2, label='Azar (AUC=0.5)')

    plt.xlabel('False Positive Rate', fontsize=12)
    plt.ylabel('True Positive Rate', fontsize=12)
    plt.title('Curvas ROC - Comparación de Modelos',
              fontsize=14, fontweight='bold')
    plt.legend(loc='lower right')
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig('roc_comparison.png', dpi=300, bbox_inches='tight')
    print("Curvas ROC guardadas como 'roc_comparison.png'")

# Preparar modelos para comparación (clasificación binaria)
modelos_binarios = {
    'Logistic Regression': LogisticRegression(random_state=42),
    'KNN (k=5)': KNeighborsClassifier(n_neighbors=5),
    'Decision Tree': DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
}

# Entrenar modelos
for modelo in modelos_binarios.values():
```

```
    modelo.fit(X_train, y_train)

# Calcular y graficar ROC
calcular_roc_multimodelo(modelos_binarios, X_test, y_test)
```

7.2 Tabla comparativa de métricas

```
# Compilar todas las métricas
print("\n\nTABLA COMPARATIVA DE MODELOS")
print("=" * 70)

resultados_comparacion = []

for nombre, modelo in modelos_binarios.items():
    # Predicciones
    y_pred = modelo.predict(X_test)

    # Probabilidades
    if hasattr(modelo, 'predict_proba'):
        probs = modelo.predict_proba(X_test)[:, 1]
    else:
        probs = modelo.decision_function(X_test)

    # Métricas
    from sklearn.metrics import precision_score, recall_score, f1_score

    acc = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    # ROC AUC
    fpr, tpr, _ = roc_curve(y_test, probs)
    roc_auc = auc(fpr, tpr)

    resultados_comparacion.append({
        'Modelo': nombre,
        'Accuracy': acc,
        'Precision': precision,
        'Recall': recall,
        'F1-Score': f1,
        'AUC': roc_auc
    })

# Crear DataFrame
df_comparacion = pd.DataFrame(resultados_comparacion)
df_comparacion = df_comparacion.round(4)
```

```
print(df_comparacion.to_string(index=False))

# Guardar
df_comparacion.to_csv('comparacion_modelos.csv', index=False)
print("\n\nTabla guardada como 'comparacion_modelos.csv'")
```

8 Aplicaciones económicas

8.1 Aplicación 1: Clasificación de riesgo crediticio

```
print("\n\n\nAPLICACIÓN: CLASIFICACIÓN DE RIESGO CREDITICIO")
print("=" * 70)

# Generar datos sintéticos
np.random.seed(42)
n_clientes = 2000

datos_credito = pd.DataFrame({
    'edad': np.random.randint(18, 70, n_clientes),
    'ingreso_anual': np.random.uniform(10000, 150000, n_clientes),
    'monto_solicitado': np.random.uniform(5000, 200000, n_clientes),
    'antiguedad_laboral': np.random.randint(0, 30, n_clientes),
    'num_creditos_previos': np.random.randint(0, 10, n_clientes),
    'ratio_deuda_ingreso': np.random.uniform(0, 1.5, n_clientes),
    'tiene_hipoteca': np.random.choice([0, 1], n_clientes),
    'score_crediticio': np.random.randint(300, 850, n_clientes)
})

# Variable objetivo: riesgo (0=bajo, 1=medio, 2=alto)
score_riesgo = (
    -datos_credito['score_crediticio'] * 0.003 +
    datos_credito['ratio_deuda_ingreso'] * 2 +
    -datos_credito['antiguedad_laboral'] * 0.05 +
    datos_credito['num_creditos_previos'] * 0.2 +
    np.random.normal(0, 0.5, n_clientes)
)

# Clasificar en 3 niveles
datos_credito['riesgo'] = pd.cut(
    score_riesgo,
    bins=3,
    labels=['Bajo', 'Medio', 'Alto']
)

# Convertir a numérico
datos_credito['riesgo_num'] = datos_credito['riesgo'].map({
    'Bajo': 0,
```

```
'Medio': 1,
'Alto': 2
})

print(f"\nDataset de crédito:")
print(f"  Total clientes: {n_clientes:,}")
print(f"\nDistribución de riesgo:")
print(datos_credito['riesgo'].value_counts())

# Preparar datos
X_credito = datos_credito.drop(['riesgo', 'riesgo_num'], axis=1)
y_credito = datos_credito['riesgo_num']

X_train_cr, X_test_cr, y_train_cr, y_test_cr = train_test_split(
    X_credito, y_credito, test_size=0.3, random_state=42, stratify=y_credito
)

# Estandarizar
scaler_credito = StandardScaler()
X_train_cr_scaled = scaler_credito.fit_transform(X_train_cr)
X_test_cr_scaled = scaler_credito.transform(X_test_cr)

# Entrenar modelos
print("\n\nENTRENAMIENTO DE MODELOS")
print("=" * 70)

modelos_credito = {
    'Random Forest': RandomForestClassifier(n_estimators=200, max_depth=10, random_st
    'Decision Tree': DecisionTreeClassifier(max_depth=8, random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=10),
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42)
}

for nombre, modelo in modelos_credito.items():
    # Entrenar
    if nombre in ['KNN', 'Logistic Regression']:
        modelo.fit(X_train_cr_scaled, y_train_cr)
        y_pred = modelo.predict(X_test_cr_scaled)
    else:
        modelo.fit(X_train_cr, y_train_cr)
        y_pred = modelo.predict(X_test_cr)

    # Evaluar
    acc = accuracy_score(y_test_cr, y_pred)

    print(f"\n{nombre}:")
    print(f"  Accuracy: {acc:.4f} ({acc*100:.1f}%)")
```

```

# Reporte por clase
print(f"\n Reporte de clasificación:")
print(classification_report(
    y_test_cr, y_pred,
    target_names=['Bajo', 'Medio', 'Alto'],
    digits=3
))

# Importancia de características (Random Forest)
print("\n\nIMPORTANCIA DE CARACTERÍSTICAS (Random Forest)")
print("=" * 70)

rf_final = modelos_credito['Random Forest']
importancias = pd.DataFrame({
    'caracteristica': X_credito.columns,
    'importancia': rf_final.feature_importances_
}).sort_values('importancia', ascending=False)

for _, row in importancias.iterrows():
    print(f" {row['caracteristica']:<25} {row['importancia']:.4f} {' ' * int(row['imp

```

8.2 Aplicación 2: Segmentación de clientes

```

print("\n\n\nAPLICACIÓN: SEGMENTACIÓN DE CLIENTES")
print("=" * 70)

# Generar datos de clientes
np.random.seed(42)
n_clientes_seg = 1500

datos_clientes = pd.DataFrame({
    'edad': np.random.randint(18, 80, n_clientes_seg),
    'ingreso_mensual': np.random.uniform(1000, 20000, n_clientes_seg),
    'gasto_mensual': np.random.uniform(500, 15000, n_clientes_seg),
    'num_compras_mes': np.random.randint(0, 30, n_clientes_seg),
    'antiguedad_meses': np.random.randint(1, 120, n_clientes_seg),
    'ticket_promedio': np.random.uniform(20, 500, n_clientes_seg),
    'visitas_web': np.random.randint(0, 100, n_clientes_seg)
})

# Calcular características adicionales
datos_clientes['frecuencia_compra'] = datos_clientes['num_compras_mes'] / 4
datos_clientes['ratio_gasto_ingreso'] = datos_clientes['gasto_mensual'] / datos_clientes['ingreso_mensual']

# Crear segmentos (0=Bronce, 1=Plata, 2=Oro, 3=Platino)
score_valor = (

```



```
    datos_clientes['gasto_mensual'] * 0.0003 +
    datos_clientes['num_compras_mes'] * 0.1 +
    datos_clientes['ticket_promedio'] * 0.005 +
    np.random.normal(0, 0.3, n_clientes_seg)
)

datos_clientes['segmento'] = pd.cut(
    score_valor,
    bins=4,
    labels=['Bronce', 'Plata', 'Oro', 'Platino']
)

datos_clientes['segmento_num'] = datos_clientes['segmento'].map({
    'Bronce': 0,
    'Plata': 1,
    'Oro': 2,
    'Platino': 3
})

print(f"\nDistribución de segmentos:")
print(datos_clientes['segmento'].value_counts().sort_index())

# Preparar datos
features_seg = ['edad', 'ingreso_mensual', 'gasto_mensual', 'num_compras_mes',
                'antiguedad_meses', 'ticket_promedio', 'visitas_web',
                'frecuencia_compra', 'ratio_gasto_ingreso']

X_seg = datos_clientes[features_seg]
y_seg = datos_clientes['segmento_num']

X_train_seg, X_test_seg, y_train_seg, y_test_seg = train_test_split(
    X_seg, y_seg, test_size=0.3, random_state=42, stratify=y_seg
)

# Entrenar Random Forest (mejor modelo típicamente)
print("\n\nMODELO DE SEGMENTACIÓN")
print("=" * 70)

rf_segmentacion = RandomForestClassifier(
    n_estimators=200,
    max_depth=12,
    min_samples_split=20,
    random_state=42
)

rf_segmentacion.fit(X_train_seg, y_train_seg)
```

```
# Evaluar
y_pred_seg = rf_segmentacion.predict(X_test_seg)
acc_seg = accuracy_score(y_test_seg, y_pred_seg)

print(f"Accuracy: {acc_seg:.4f} ({acc_seg*100:.1f}%)")

# Matriz de confusión
cm_seg = confusion_matrix(y_test_seg, y_pred_seg)
print(f"\nMatriz de confusión:")
print(f"
          Predicho")
print(f"
      Bronce  Plata  Oro  Platino")
print(f"Real Bronce  {cm_seg[0, 0]:4d}  {cm_seg[0, 1]:4d}  {cm_seg[0, 2]:4d}  {cm_seg[0, 3]:4d}")
print(f"      Plata  {cm_seg[1, 0]:4d}  {cm_seg[1, 1]:4d}  {cm_seg[1, 2]:4d}  {cm_seg[1, 3]:4d}")
print(f"      Oro  {cm_seg[2, 0]:4d}  {cm_seg[2, 1]:4d}  {cm_seg[2, 2]:4d}  {cm_seg[2, 3]:4d}")
print(f"      Platino  {cm_seg[3, 0]:4d}  {cm_seg[3, 1]:4d}  {cm_seg[3, 2]:4d}  {cm_seg[3, 3]:4d}")

# Características más importantes
print(f"\n\nCARACTERÍSTICAS MÁS IMPORTANTES PARA SEGMENTACIÓN")
print("=" * 70)

importancias_seg = pd.DataFrame({
    'caracteristica': features_seg,
    'importancia': rf_segmentacion.feature_importances_
}).sort_values('importancia', ascending=False)

for _, row in importancias_seg.iterrows():
    barra = ' ' * int(row['importancia'] * 100)
    print(f" {row['caracteristica']:<25} {row['importancia']:.4f} {barra}")

print(f"\n\nAPLICACIÓN PRÁCTICA:")
print("Este modelo permite:")
print(" 1. Clasificar automáticamente nuevos clientes")
print(" 2. Personalizar ofertas según segmento")
print(" 3. Identificar clientes con potencial de upgrade")
print(" 4. Optimizar estrategias de retención")
```

9 Ejercicios prácticos

9.1 Ejercicio: Optimización de hiperparámetros

```
print("\n\n\nEJERCICIO: OPTIMIZACIÓN DE HIPERPARÁMETROS")
print("=" * 70)

# Grid search manual para Random Forest
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 15, None],
```

```
'min_samples_split': [2, 10, 20]
}

mejores_params = None
mejor_score = 0

print("\nBuscando mejores hiperparámetros...")
print(f"Total combinaciones: {len(param_grid['n_estimators']) * len(param_grid['max_d

resultados_grid = []

for n_est in param_grid['n_estimators']:
    for max_d in param_grid['max_depth']:
        for min_split in param_grid['min_samples_split']:

            # Entrenar modelo
            rf = RandomForestClassifier(
                n_estimators=n_est,
                max_depth=max_d,
                min_samples_split=min_split,
                random_state=42
            )

            # Validación cruzada
            scores = cross_val_score(rf, X_train, y_train, cv=5)
            score_mean = scores.mean()

            resultados_grid.append({
                'n_estimators': n_est,
                'max_depth': max_d if max_d else 'None',
                'min_samples_split': min_split,
                'cv_score': score_mean
            })

            if score_mean > mejor_score:
                mejor_score = score_mean
                mejores_params = {
                    'n_estimators': n_est,
                    'max_depth': max_d,
                    'min_samples_split': min_split
                }

print(f"\n\nMEJORES HIPERPARÁMETROS")
print("=" * 70)
print(f"  n_estimators: {mejores_params['n_estimators']}")
print(f"  max_depth: {mejores_params['max_depth']}")
print(f"  min_samples_split: {mejores_params['min_samples_split']}")
```

```
print(f"  CV Score: {mejor_score:.4f}")

# Entrenar modelo final
rf_optimo = RandomForestClassifier(**mejores_params, random_state=42)
rf_optimo.fit(X_train, y_train)

acc_optimo = rf_optimo.score(X_test, y_test)
print(f"\nAccuracy en test: {acc_optimo:.4f} ({acc_optimo*100:.1f}%)")

# Top 10 combinaciones
df_grid = pd.DataFrame(resultados_grid).sort_values('cv_score', ascending=False)
print(f"\n\nTop 10 combinaciones:")
print(df_grid.head(10).to_string(index=False))
```

10 Conclusión

En esta guía hemos explorado los principales algoritmos de clasificación:

Regresión Logística

Modelo lineal simple y rápido. Usa función sigmoide para probabilidades. Interpretable.

Bueno como baseline.

K-Nearest Neighbors (KNN)

No paramétrico. Clasifica según vecinos más cercanos. Sensible a escala. Bueno para fronteras complejas.

Árboles de Decisión

Reglas interpretables. No requiere normalización. Propenso a overfitting. Base para métodos ensemble.

Random Forests

Ensemble de árboles. Reduce overfitting. Alta precisión. Menos interpretable que árbol único.

Comparación de modelos

Importancia de evaluar múltiples algoritmos. Usar validación cruzada. Considerar trade-offs interpretabilidad vs precisión.

11 Próximos pasos

En la siguiente guía (Guía 10: Métodos de ML para Regresión) exploraremos:

- Regresión lineal
- Ridge y Lasso
- Regresión polinomial
- Métricas: R^2 , MSE, MAE
- Aplicaciones económicas

12 Recursos adicionales

Para profundizar en clasificación:

- Scikit-learn Documentation: scikit-learn.org
- Introduction to Statistical Learning (ISLR)
- Hands-On Machine Learning (Aurélien Géron)
- Papers en credit scoring y risk management

13 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1.  [Instalacion De Anaconda](#)
2.  [Configurar Entorno Virtual Python Anaconda](#)
3.  [01 Introducion A La Programacion Con Python](#)
4.  [02 Variables Expresiones Y Statements Con Python](#)
5.  [03 Objetos De Python](#)
6.  [04 Ejecucion Condicional Con Python](#)
7.  [05 Iteraciones Con Python](#)
8.  [06 Funciones Con Python](#)
9.  [07 Dataframes Con Python](#)
10.  [08 Prediccion Y Metrica De Performance Con Python](#)
11.  [09 Metodos De Machine Learning Para Clasificacion Con Python](#)
12.  [10 Metodos De Machine Learning Para Regresion Con Python](#)
13.  [11 Validacion Cruzada Y Composicion Del Modelo Con Python](#)
14.  [Visualizacion De Datos Con Python](#)

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!